00_ENGINEERING_RULES.md

ENGINEERING RULES

Version: 1.0

Status

Mandatory

Applies To

Entire Project

Every engineer

Every feature

Every file

Every commit

Every document

Every review

Purpose

This document defines the engineering standards for the Secure Dance Academy Management System.

Every contributor must follow these rules.

If a conflict exists between implementation speed and engineering quality, engineering quality always wins.

If a conflict exists between usability and security, find the safest solution that maintains a good user experience.

Never ignore these rules.

Engineering Mindset

Build software for production.

Do not build software only to satisfy coursework.

Every feature should be deployable.

Every decision should be explainable.

Every engineer is responsible for quality.

Think long term.

Future developers should understand your work without asking questions.

Decision Priority

When choosing between multiple solutions, follow this order.

1 Security

2 Correctness

3 Maintainability

4 Simplicity

5 Scalability

6 Performance

7 Developer Convenience

8 Visual Effects

Never reverse this order.

Code Quality Rules

Write readable code.

Use descriptive names.

Keep functions focused.

Keep components small.

Avoid duplicated logic.

Avoid deeply nested conditions.

Prefer composition.

Prefer reusable code.

Separate concerns.

Follow consistent formatting.

Write code that explains itself.

Architecture Rules

Follow Feature Based Clean Architecture.

Separate:

Presentation

Business Logic

Data Access

Infrastructure

Utilities

Business logic must never exist inside UI components.

Database access must never exist inside presentation components.

Security logic must never be duplicated.

Folder Rules

Every feature owns its own files.

Authentication

Artists

Parents

Coaches

Attendance

Performance

Injuries

Reports

Settings

Audit

Avoid dumping unrelated code into shared folders.

Organize by feature before organizing by file type.

API Rules

Every endpoint must include

Authentication

Authorization

Validation

Business Logic

Error Handling

Audit Logging where appropriate

Consistent Responses

Never expose internal implementation.

Never expose unnecessary data.

Never skip authorization.

Database Rules

Use Prisma ORM.

Never bypass the ORM without approval.

Normalize the schema.

Use foreign keys.

Use indexes.

Use transactions.

Protect referential integrity.

Use UUIDs.

Use soft deletes where appropriate.

Maintain audit history.

Security Rules

Authentication is mandatory.

Authorization is mandatory.

Validation is mandatory.

Rate limiting is mandatory.

Audit logging is mandatory.

Protect sensitive information.

Use environment variables.

Never expose secrets.

Never trust client input.

Always validate on the server.

Never rely on frontend authorization.

Follow:

OWASP Top 10

OWASP ASVS

NIST Secure Software Development Framework

Authentication Rules

Preferred

Supabase Authentication

Alternative

JWT using Secure HTTP-only Cookies

Never store tokens inside localStorage.

Never store passwords.

Always hash passwords.

Terminate sessions securely.

Protect password reset functionality.

Validation Rules

Validate:

Request Body

Query Parameters

Route Parameters

Headers

Uploaded Files

Use Zod.

Reject invalid input immediately.

Validation should exist on both client and server.

Error Handling Rules

Return meaningful messages.

Never expose:

Stack traces

Database queries

Framework internals

Secrets

Return consistent response structures.

Log detailed errors internally.

Logging Rules

Log:

Authentication

Authorization

Attendance

Medical Updates

Role Changes

Administrative Actions

Security Events

Audit events should be immutable.

Logs should support investigation.

UI Rules

Every page must include

Loading State

Error State

Empty State

Responsive Layout

Accessible Navigation

Validation Feedback

Confirmation Dialogs

Professional design is mandatory.

Accessibility Rules

Meet WCAG 2.2 AA.

Support:

Keyboard Navigation

Screen Readers

Focus Indicators

Semantic HTML

Accessible Forms

High Contrast

Accessibility defects are quality defects.

Performance Rules

Optimize:

Rendering

Database Queries

Search

Filtering

Pagination

Bundle Size

Images

Avoid unnecessary requests.

Optimize before deployment.

Documentation Rules

Every feature requires documentation.

Every API requires documentation.

Every security decision requires justification.

Every diagram requires explanation.

Every architecture decision should be recorded.

Documentation should remain synchronized with implementation.

Testing Rules

Every feature requires:

Unit Tests

Integration Tests

API Tests

Security Tests

Regression Tests

Critical workflows require End-to-End Tests.

No feature is complete without testing.

Git Rules

Commit frequently.

One logical change per commit.

Write meaningful commit messages.

Examples

feat: implement attendance module

fix: validate injury recovery dates

security: restrict parent access

Avoid vague commit messages.

Pull Request Rules

Before merging verify:

Feature Complete

Tests Pass

Documentation Updated

Security Reviewed

No Critical Issues

Architecture Preserved

Code Reviewed

Never merge unfinished work.

Technical Debt Rules

Do not ignore technical debt.

Record it.

Prioritize it.

Resolve it before release whenever practical.

Avoid temporary fixes becoming permanent.

AI Collaboration Rules

Every AI agent must:

Read Project Context.

Read its assigned handbook.

Understand previous work.

Respect architectural decisions.

Avoid rewriting completed modules.

Avoid introducing inconsistency.

Review its own output before completion.

Improve existing work rather than replacing it.

Definition of Done

Work is complete only when:

Requirements are satisfied.

Implementation is complete.

Security is implemented.

Validation is complete.

Tests pass.

Documentation is updated.

Code review passes.

No critical defects remain.

The feature integrates cleanly with the existing system.

Definition of Excellence

Excellent engineering produces software that remains understandable, secure and maintainable long after development ends.

Every decision should improve the overall system.

Every feature should contribute measurable value.

Every engineer should leave the project in a better state than they found it.

Professional software is not measured by the amount of code written.

It is measured by the quality of the decisions behind that code.